PROJECT REPORT

ON

Design and Implementation of Bidirectional Inter-Process Communication Using Pipes

Submitted to

NMAM INSTITUTE OF TECHNOLOGY, NITTE

(Off-Campus Centre, Nitte Deemed to be University, Nitte - 574 110, Karnataka, India)

In partial fulfilment of the requirements for the award of the

Degree of Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

by

Diya Sajin	NNM24CB022
Varashri R H	NNM24CB068

Under the guidance of

S Shyam Kumar

Assistant Professor

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

2025 – 2026

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Department of Cyber Security

CERTIFICATE

This is to certify that Ms. Diya Sajin (NNM24CB022)and Varashri R H (NNM24CB068) of II-year B.Tech., a bonafide students of NMAM Institute of Technology, Nitte,has carried out project on "**Design and Implementation of Bidirectional Inter-Process Communication Using Pipes**" as part of the **Unix System Programming and Networking Lab (CB2601-1)** course during 2025-26, fulfilling the partial requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security) at NMAM Institute of Technology, Nitte.

…….....

Name and Signature of Course Instructor

ABSTRACT

Inter-Process Communication (IPC) is a fundamental concept in Unix-based operating systems that enables multiple processes to communicate and share data efficiently. This project focuses on implementing a simple chat system between two processes using pipes, which are one of the most basic IPC mechanisms.

The main objective of this project is to demonstrate how a parent process and a child process can exchange messages using a unidirectional communication channel. The system utilizes core Unix system calls such as pipe(), fork(), read(), and write() to establish communication between processes.

In this implementation, one process writes messages into the pipe while the other process reads and displays them, thereby simulating a simple chat mechanism. This approach highlights how processes can work cooperatively while maintaining independence.

The project also emphasizes concepts such as process creation, synchronization, and data flow management. The result is a simple yet effective demonstration of IPC, which can be further extended into more complex communication systems used in real-world applications such as messaging systems, client-server architectures, and distributed computing.

LIST OF CONTENTS

ABSTRACT **i**

CHAPTERS **Page No.**

1. INTRODUCTION **1**

1.1 Purpose

1.2 Scope

1.3 Overview

2. REQUIREMENT SPECIFICATIONS **2**

2.1 Hardware Specification

2.2 Software Specification

3. SYSTEM DESIGN **3**

3.1 Architecture Diagram

3.2 Working Principle

3.3 Device Role Assignment

4. IMPLEMENTATION **4**

4.1 Pipe Creation

4.2 Process Creation using fork()

4.3 Writing to Pipe

4.4 Laptop Configuration (WI-FI Client)

4.5 Server Configuration (HTTP Host)

5. RESULTS AND DISCUSSION **5**

5.1 Testing environment

5.2 Ping test Result

5.3 Website Hosting Test

6. CONCLUSION AND FUTURE WORK **6**

6.1 Conclusion

6.2 Future work

REFERENCES **7**

CHAPTER 1
INTRODUCTION

1.1 Purpose

The primary purpose of this project is to understand and implement inter-process communication using pipes in a Unix environment. Processes in an operating system often need to exchange information, and pipes provide a simple and efficient way to achieve this.

This project aims to simulate a basic chat system between two processes, where one process sends messages and the other receives them. It helps in understanding how processes can coordinate and communicate without sharing memory directly.

1.2 Scope

The scope of this project includes:

- Understanding IPC concepts
- Creating processes using fork()
- Implementing communication using pipes
- Sending and receiving messages between processes
- Demonstrating unidirectional communication

This project is limited to communication between two processes on the same system and does not include network-based communication.

1.3 Overview

In this project, a pipe is created to enable communication between two processes. After creating the pipe, the process is divided into parent and child using the fork() system call.

The parent process sends messages through the pipe, while the child process reads those messages and displays them. This demonstrates how data flows between processes in a controlled and efficient manner.

The system showcases the working of Unix system calls and provides a practical understanding of IPC mechanisms.

CHAPTER 2

REQUIREMENT SPRECIFICATION

2.1 Hardware Specification

- Cisco Router (2811)
- Wireless Router (WRT300N)
- Cisco Switch (2960)
- 6 PCs
- 1 Laptop
- 1 Generic Server
- Ethernet Cables (Straight-through)

2.2 Software Specification

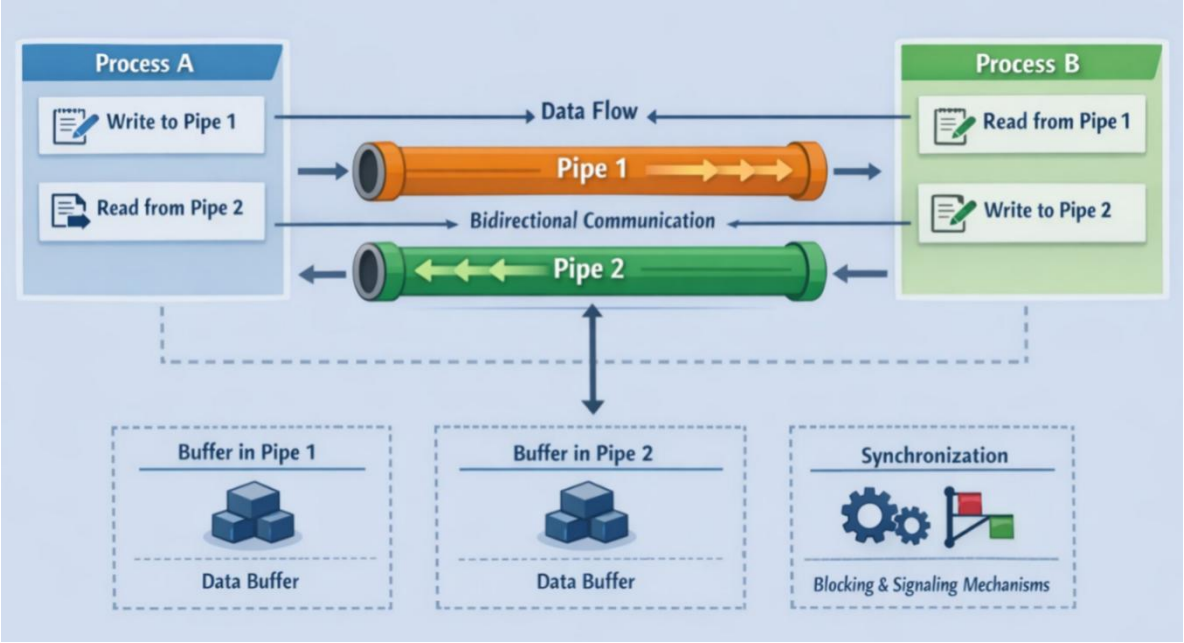
- Operating System: Linux / Unix (Ubuntu preferred)
- Programming Language: C
- Compiler: GCC (GNU Compiler Collection)
- Terminal for execution

Additional tools such as text editors (VS Code, Nano, or Vim) can be used for writing the program.

CHAPTER 3

SYSTEM DESIGN

3.1 Architectural Diagram



3.2 Working Principle

A pipe is created using the pipe() system call, which provides two file descriptors:

- fd[0] → Read end
- fd[1] → Write end

After calling fork(), two processes are created:

- The parent process writes data into the pipe
- The child process reads data from the pipe

The communication is unidirectional, meaning data flows only in one direction.

3.3 Proper Communication Flow

1. Create pipe
2. Call fork()
3. Parent writes message
4. Child reads message
5. Output is displayed

This flow ensures proper synchronization between processes.

CHAPTER 4

IMPLEMENTATION

4.1 Pipe Creation

The pipe is created using:

```
int fd[2];
pipe(fd);
```

This creates two ends for communication.

4.2 Process Creation using fork()

```
pid_t pid = fork();
```

- If pid > 0 → Parent process
- If pid == 0 → Child process

4.3 Writing to Pipe

The parent process writes data:

```
write(fd[1], message, strlen(message) + 1);
```

4.4 Reading from Pipe

The child process reads data:

```
read(fd[0], buffer, sizeof(buffer));
```

4.5 Complete Program Explanation

- Pipe is initialized
- Process is forked
- Parent sends message
- Child receives message
- Output is printed

This demonstrates successful IPC.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Testing Environment

The program was tested in a Linux environment using GCC compiler. Execution was done through terminal.

5.2 Output Results

Example output:

Parent: Enter message for child: Hello

Child received: Hello

Child: Enter reply to parent: Hi

Parent received: Hi

5.3 Analysis

- Communication was successful
- No data loss observed
- Execution was fast and efficient
- Demonstrates reliability of pipes

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully demonstrates inter-process communication using pipes. It shows how processes can exchange data efficiently using system calls.

The implementation helps in understanding core operating system concepts such as process creation and communication.

6.2 Future Scope

The project can be extended by:

- Implementing two-way communication using two pipes
- Adding multiple processes
- Creating a real-time chat system
- Using sockets for network communication
- Adding GUI interface

REFERENCES

1. Unix System Programming Textbook

Author: W. Richard Stevens

Book: Advanced Programming in the UNIX Environment

2. Operating System Concepts

Authors: Abraham Silberschatz, Peter B. Galvin

Topic referred: Processes and Inter-Process Communication